UNDERSTANDING GUIDE

PlaceIQ

Everything the team needs to understand, present and defend the project

Fundamentals of Database Systems (FDS)

Prof. Debapriyay Mukhopadhyay · PGDBA Batch 12, IIM Calcutta

Member	Name	Presentation segment
M1	Ekta Sharma	Problem & EER
M2	[Member 2]	Schema & normalization
M3	[Member 3]	DDL, data & queries
M4	[Member 4]	Rules, transactions & performance

November 2026 · github.com/EktaSharma-github/placeiq

Contents

1. How to use this guide 3
2. The project in two minutes 4
3. Where everything lives 5
   3.1 Build order 5
4. Concepts, mapped to PlaceIQ 6
5. What happens when someone clicks 'Accept' 8
6. Question bank 9
   6.1 M1 · Ekta Sharma · Problem & EER 9
   6.2 M2 · [Member 2] · Schema & normalization 9
   6.3 M3 · [Member 3] · DDL, data & queries 10
   6.4 M4 · [Member 4] · Rules, transactions & performance 10
   6.5 Questions anyone may get 11
7. Personal preparation sheets 12
   M1 · Ekta Sharma · Problem & EER 12
   M2 · [Member 2] · Schema & normalization 12
   M3 · [Member 3] · DDL, data & queries 12
   M4 · [Member 4] · Rules, transactions & performance 12
8. Glossary 14
9. Numbers cheat sheet 15

(If page numbers are missing, right-click the table and choose Update Field.)

1. How to use this guide

This guide is for the four of us, not for the examiner. Read sections 2 and 3 once to get the whole picture. Then use section 4 as a concept-by-concept reference: for each course topic it says what the concept is, where it appears in PlaceIQ, and one sentence you can say aloud. Section 6 is the question bank; section 7 is your personal preparation sheet.

The one sentence to remember

PlaceIQ replaces the placement committee's spreadsheet with a PostgreSQL database that enforces the placement rules itself: ineligible applications, double-booked interviews and second acceptances are refused by the database, not by people.

2. The project in two minutes

1. **Problem.** The committee uses a spreadsheet; it duplicates data, suffers anomalies and cannot enforce rules.
2. **Model.** We drew an EER model with every advanced construct: overlapping and disjoint specialization, a weak entity, aggregation and a union type.
3. **Schema.** We mapped it to 22 tables and proved them to be in BCNF from the functional dependencies.
4. **Rules.** Constraints enforce what can be declared; triggers enforce rules that span tables; a stored procedure accepts offers atomically.
5. **Data.** 180 students and 1,184 applications, generated reproducibly and loaded through the rules.
6. **Questions.** Five queries answer real committee questions using HAVING, division, correlated subqueries and window functions.
7. **Proof.** 91 automated tests pass; indexing, concurrency and SQL injection were measured and demonstrated.
8. **Show.** A web UI lets us demonstrate it live, with a recorded backup video.

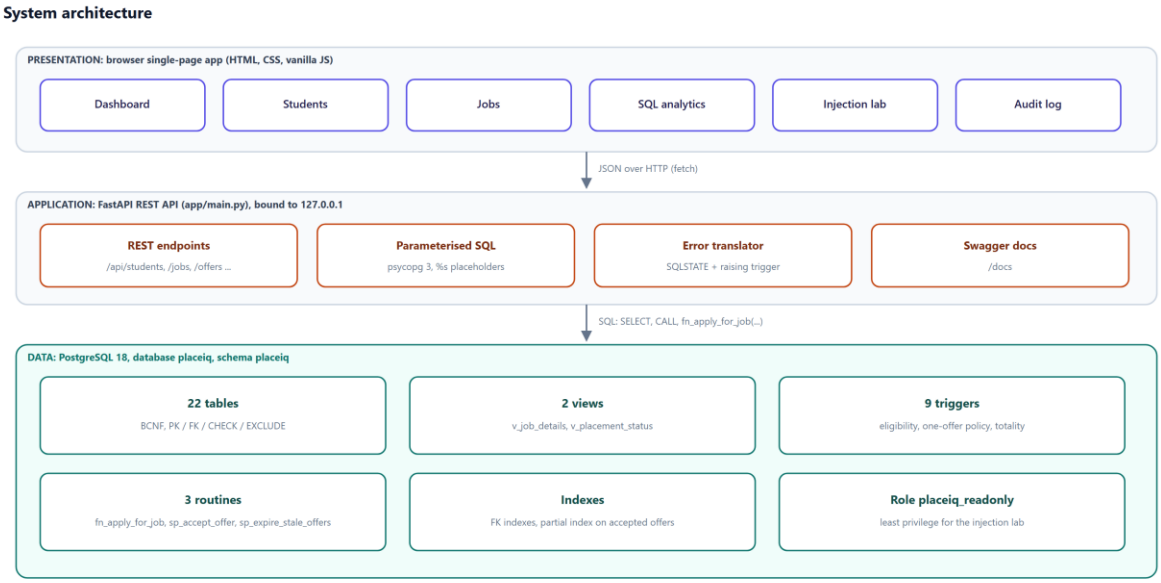


Figure 1. *How the parts fit: browser → REST API → PostgreSQL.*

3. Where everything lives

File	What it contains	Who presents it
01_design.md	Business rules, EER description, mapping, normalization notes	M1, M2
02_ddl.sql	CREATE TABLE statements, constraints, FK indexes	M3
03_logic.sql	Views, triggers, procedures, the read-only role	M4
04_data.sql	Mock data and the recruitment simulation	M3
05_queries.sql	The five analytical queries	M3
06_demo.sql	psql demo script (fallback for the live demo)	M4
app/	Web UI and REST API	M4 (demo)
tests/	Automated tests and results	M4
deliverables/	Report, this guide, testing report, runbook, slides, video, diagrams	All

3.1 Build order

02_ddl.sql creates empty tables → 03_logic.sql installs views, triggers and procedures → 04_data.sql loads data through those triggers in one transaction. run_all.ps1 does all three; the UI's 'Reset demo data' button does the same.

4. Concepts, mapped to PlaceIQ

Each row: the textbook idea, where it is in our project, and a sentence you can say.

Concept	What it is	Where in PlaceIQ	Say it like this
DBMS vs file system	A DBMS manages shared data with integrity, concurrency, recovery and security; a file just stores bytes.	The spreadsheet problem in section 1: duplication, anomalies, no rule enforcement.	"We moved the rules from people into the DBMS."
Data models	Ways to describe data: hierarchical, network, relational, object-oriented.	We use the relational model; the EER model is our conceptual model.	"Relational: everything is a table of tuples with keys."
Entity, attribute types	Simple/composite, single/multi-valued, stored/derived, key.	name is composite; skills is multi-valued; placement_status is derived.	"Derived attributes are computed, not stored: our view does that."
Cardinality & participation	How many instances relate (1:1, 1:N, M:N); whether participation is total or partial.	Student–Program N:1 with total participation (double line).	"Every student must be in a program: total participation."
Specialization / generalization	Subclasses of a superclass; disjoint (d) or overlapping (o); total or partial.	PERSON → STUDENT… overlapping, partial. JOB_ROLE → INTERNSHIP / FULLTIME disjoint, total.	"A student can also be on the committee, so it's overlapping."
Weak entity	An entity with no key of its own; identified through its owner via an identifying relationship.	INTERVIEW_ROUND, key (job_id, round_no); round_no is the partial key.	"'Round 2' means nothing without its job."
Aggregation	Treating a relationship (with its entities) as an entity so it can take part in another relationship.	APPLIES_TO is aggregated; interviews and offers attach to an application.	"You interview an application, not a student in general."
Category / union type	A subclass whose superclasses are of different entity types.	SPONSOR = COMPANY ∪ ALUMNUS.	"Two different kinds of thing playing the same role."
ER → relational mapping	Standard steps converting each construct to tables and keys.	22 tables; 8A for subclasses; surrogate key for the category.	"One table per subclass, so subtype columns can be NOT NULL."
Keys	Super key ⊇ candidate key (minimal) → primary key; others are alternate keys; foreign keys reference keys.	placement_cycle has 3 candidate keys; job natural key kept UNIQUE.	"Surrogate ids for convenience, natural keys kept as UNIQUE."
Integrity constraints	Domain (CHECK), entity (PK), referential (FK), key (UNIQUE), plus general constraints.	45 CHECK, 30 FK, 17 UNIQUE, 1 EXCLUDE.	"If a rule can be declared, we declared it."
Functional dependency	X → Y: equal X values imply equal Y values.	roll_no → name, cgpa; prog_code → prog_name; (job, program) → min_cgpa.	"FDs are the business facts that decide the tables."
Anomalies	Update, insertion and deletion problems caused by redundancy.	Company HQ repeated; job can't exist without an applicant.	"Normalization exists to remove these three."
1NF / 2NF / 3NF / BCNF	Atomic values / no partial dependency / no transitive dependency / every determinant a super key.	skills split (1NF); student, company… split (2NF); program, eligibility split (3NF).	"Every determinant in every table is a key, so BCNF."
Lossless & dependency-preserving	Join of parts gives back the original; every FD checkable in one table.	Heath's theorem on each split; FD1–FD9 each live in one table.	"We lost no data and no rule."
DDL / DML / DQL	Define structure / change data / query data.	02_ddl.sql / 04_data.sql / 05_queries.sql.	"Three files, three sub-languages."
Joins & outer joins	Combine tables on a condition; outer joins keep unmatched rows.	Q1 LEFT JOIN offer (not every application has an offer).	"Outer join so applicants without offers still count."
Aggregation & HAVING	GROUP BY summarises; HAVING filters groups (WHERE filters rows).	Q1: companies whose offer rate is below the campus average.	"HAVING compares a group to a scalar subquery."
Nested & correlated subqueries	A query inside a query; correlated ones reference the outer row and re-run per row.	Q5 compares each student to the average of that job's selected students.	"Correlated: it re-evaluates for each application."

Concept	What it is	Where in PlaceIQ	Say it like this
Relational division	Find X related to ALL Y. No SQL operator; use double NOT EXISTS or counting.	Q2: students holding every skill a job needs.	"No required skill that the student lacks."
Views	Stored queries that act like tables.	v_job_details; v_placement_status (derived attribute).	"Status is a view, so it can never be wrong."
Window functions	Aggregates computed over a partition without collapsing rows.	Q3 DENSE_RANK, NTILE, AVG OVER; Q4 running SUM OVER.	"Ranks and running totals while keeping every row."
Triggers	Code the DBMS runs automatically before/after a change.	Eligibility on apply; one-offer cascade; totality check.	"For rules that span several tables."
Stored procedures / functions	Named server-side routines.	fn_apply_for_job, sp_accept_offer, sp_expire_stale_offers.	"Single, parameterised entry points."
Transactions & ACID	Atomicity, Consistency, Isolation, Durability.	Accept + cascade in one transaction; ROLLBACK undoes all.	"All of it happens, or none of it."
Concurrency & locking	Isolation levels; row locks (SELECT … FOR UPDATE); deadlock.	Student row locked first; second session waits then fails cleanly.	"Lock order prevents deadlock."
Indexes	B-tree (and other) structures for fast lookup; composite and partial indexes.	1,527× faster with a composite index on 600k rows.	"An index lets the DBMS seek instead of scan."
Query optimisation / EXPLAIN	The planner chooses scans and joins by cost; EXPLAIN ANALYZE shows the plan and real time.	Seq Scan vs Bitmap Index Scan; blocks read.	"We measured, we didn't assume."
SQL injection	Untrusted input changing the SQL statement's structure.	Injection lab: 180 rows leaked vs 0 with a bound parameter.	"Parameters are data, never code."
DCL: roles & privileges	GRANT / REVOKE control who may do what.	placeiq_readonly: SELECT on two tables only.	"Least privilege limits the damage of any bug."
REST and databases	Web APIs expose resources over HTTP; the server talks SQL to the DBMS.	FastAPI endpoints → views, function, procedure.	"The API is thin; the database holds the rules."

5. What happens when someone clicks 'Accept'

Following one request end to end is the best way to understand how all the layers fit. The student is PGP62-084 (Dev Saxena), who holds a pending DataSutra offer.

1. **Browser.** The Accept button calls POST /api/offers/89/accept.
2. **API (main.py).** Opens a connection and runs CALL sp_accept_offer(%s) with 89 as a bound parameter.
3. **Procedure.** Finds the offer's student and locks that student row with SELECT … FOR UPDATE. Any other session accepting for the same student now waits.
4. **Procedure.** Re-reads the offer under the lock. If it is no longer PENDING, it raises an error and nothing changes.
5. **UPDATE offer SET status = 'ACCEPTED'.** This fires the BEFORE trigger trg_offer_accept_guard: deadline not passed, no other accepted offer in this cycle.
6. **AFTER trigger trg_offer_accept_cascade.** Declines the student's other pending offers, withdraws open applications, deletes their scheduled interviews for other jobs, writes rows to placement_log.
7. **COMMIT.** The API's connection commits; everything above becomes permanent together. If any step had failed, all of it would roll back.
8. **Response.** The API reads the log rows written 'now' and returns them; the browser shows the toast: 1 declined, 1 withdrawn.
9. **View.** The profile reloads; v_placement_status now computes PLACED for the final cycle.

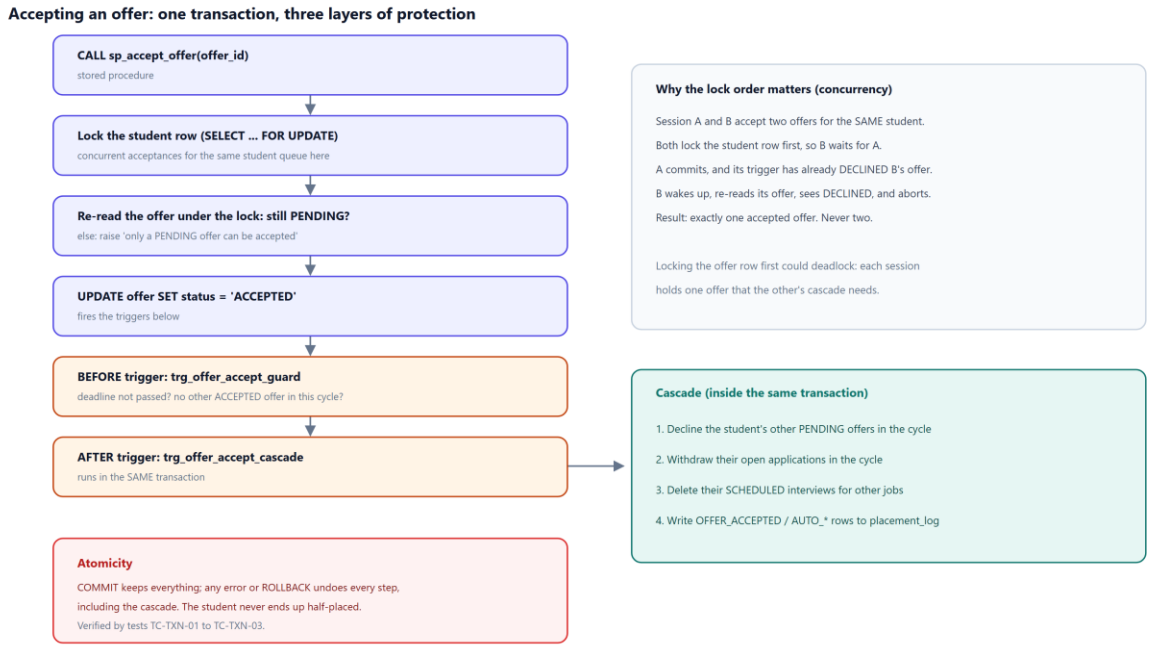


Figure 2. *The accept flow and why the lock order matters.*

6. Question bank

Grouped by the member who should answer. Answers are deliberately short: say them, then stop.

6.1 M1 · Ekta Sharma · Problem & EER

#	Question	Model answer
1	Why not just use Excel with data validation?	Validation works per cell. Our rules span rows and tables (one accepted offer per cycle, no overlapping interviews, eligibility from another table), must hold under concurrent users, and must be atomic. Those need a DBMS: constraints, triggers and transactions.
2	Why is the PERSON specialization overlapping?	A student can also be a placement committee member, so one person can belong to two subclasses. It is partial because a person (for example a faculty coordinator) need not be in any subclass.
3	Why is JOB_ROLE disjoint and total?	A job is either an internship (stipend, duration) or a full-time role (CTC); never both, never neither.
4	Why use aggregation instead of a ternary relationship?	An interview concerns an existing application, not any student-job-round combination. Aggregation makes the application the participant, so we can enforce (with foreign keys) that interviews and offers only exist for real applications. A ternary relationship would not express that dependency.
5	What makes INTERVIEW_ROUND weak?	It has no identifier of its own: round_no repeats across jobs. Its key is the owner's key plus the partial key (job_id, round_no), via the identifying relationship HAS_ROUND.
6	What is a category (union type), and why use it?	A subclass of superclasses of different types. A sponsor can be a company or an alumnus: different entity types sharing one role. We map it with a surrogate sponsor_id.
7	Why is placement_status a derived attribute?	It follows from offers. Storing it would create redundancy that could disagree with the offers table, so a view computes it.
8	Total vs partial participation, with an example?	Total: every student must be enrolled in a program (double line). Partial: a company need not be a sponsor.
9	Which business rules could not be expressed in the ER model?	Eligibility (CGPA cut-off per program), one offer per cycle and non-overlapping slots. They became CHECK/EXCLUDE constraints and triggers.
10	How did you collect the business rules?	From how the placement process works: eligibility criteria per company and program, the one-offer policy, interview scheduling. We listed 15 and mapped each to an enforcement mechanism.

6.2 M2 · [Member 2] · Schema & normalization

#	Question	Model answer
1	Define super, candidate, primary and alternate keys using your schema.	In placement_cycle, (cycle_id, cycle_name) is a super key; cycle_id, cycle_name and (cycle_type, academic_year) are candidate keys; cycle_id is primary; the others are alternate keys, kept UNIQUE.
2	Why use surrogate keys at all?	Shorter, stable, efficient joins (job_id instead of company, cycle and title). We kept every natural key as UNIQUE, so no dependency is lost.
3	Show a partial dependency in your design.	In the 1NF sheet with key (roll_no, J, round_no), roll_no → cgpa: cgpa depends on part of the key. It moves to Student in 2NF.
4	Show a transitive dependency.	roll_no → prog_code → prog_name. Program is separated in 3NF. Also (J, prog_code) → min_cgpa, giving Eligibility.
5	Is Student in BCNF even though email → roll_no?	Yes. email and roll_no are both candidate keys, so every determinant is a super key.
6	Give a relation in 3NF but not in BCNF.	Panel(student, job, recruiter) if each recruiter served one job: recruiter → job, recruiter is not a super key, but job is prime. We avoided the rule because the business does not need it.

#	Question	Model answer
7	How do you know the decomposition is lossless?	Each split on X → Y produced (XUY) and (R−Y) with X a key of the first part. By Heath's theorem the natural join reconstructs R.
8	Is it dependency-preserving?	Yes: each of FD1–FD9 is contained in a single resulting relation, so every rule can be checked locally.
9	Why not store CTC in the offer table?	offer → job_id → ctc_lpa is transitive. Pay is a fact about the job, so it lives in fulltime_role.
10	Why one table per subclass (8A)?	A single table would need NULLs in half its rows and could not declare that CTC is mandatory for full-time jobs. With 8A, subtype columns are NOT NULL.

6.3 M3 · [Member 3] · DDL, data & queries

#	Question	Model answer
1	How is disjointness enforced without a trigger?	Each subtype has a role_type column fixed by CHECK, and a composite FK to (job_id, role_type) in job_role. A full-time job has no matching key for an internship row.
2	Why an EXCLUDE constraint instead of UNIQUE?	UNIQUE only detects equal values. We must reject overlapping time ranges; EXCLUDE USING gist with && (overlaps) does that. It needs btree_gist for the = on student_id.
3	Explain ON DELETE CASCADE vs RESTRICT in your schema.	CASCADE where the child cannot exist alone (subtypes, rounds). RESTRICT to protect records (offers). Deleting a student cascades to applications, but an offer's RESTRICT blocks the whole delete.
4	Why index foreign keys manually?	PostgreSQL indexes PKs and UNIQUE constraints automatically, not FKs. Without an index, joins and ON DELETE checks scan the child table.
5	How is the mock data realistic yet reproducible?	setseed() fixes the random sequence; CGPA is triangular; shortlist probability rises with CGPA; a simulation runs the recruitment process through the real triggers.
6	Explain Q2's double NOT EXISTS.	'Student has every required skill' = 'there is no required skill the student lacks'. Outer NOT EXISTS over required skills, inner NOT EXISTS over the student's skills.
7	WHERE vs HAVING?	WHERE filters rows before grouping; HAVING filters groups after aggregation. Q1 uses HAVING to compare each company's offer rate with the campus rate.
8	DENSE_RANK vs RANK vs ROW_NUMBER?	ROW_NUMBER is always unique; RANK gives ties the same rank and skips the next (1,1,3); DENSE_RANK does not skip (1,1,2).
9	Why can't you filter on a window function in WHERE?	Window functions are computed after WHERE. We wrap the query in a subquery and filter on its result.
10	What is a correlated subquery and what is its cost?	It references the outer row and is evaluated per row (Q5 per application). It can be slower; the planner may decorrelate it, and indexes on the inner join columns help.

6.4 M4 · [Member 4] · Rules, transactions & performance

#	Question	Model answer
1	Why a trigger instead of application code for eligibility?	The rule must hold however data arrives: psql, the API or another app. A trigger in the DBMS cannot be bypassed and runs inside the same transaction.
2	BEFORE vs AFTER triggers in your project?	BEFORE validates and can reject (guard, eligibility). AFTER reacts to a committed-to-be change (cascade, marking the application OFFERED).
3	What does 'atomic' mean for accepting an offer?	Accept, decline the others, withdraw applications, delete interview slots and write the log: all in one transaction. Any error rolls back every step.
4	What happens if two sessions accept two offers at the same time?	Both lock the student row first; the second waits. When the first commits, its cascade has declined the second offer; the second session re-reads it, sees DECLINED and fails. Exactly one acceptance survives.
5	Why could the earlier version deadlock?	It locked the offer row first. A held offer 1 and needed offer 2 for its cascade; B held offer 2 and needed offer 1. Locking the student first gives one global order.

#	Question	Model answer
6	What isolation level do you use?	PostgreSQL's default, READ COMMITTED, plus explicit row locks where needed. SERIALIZABLE with retries is an alternative.
7	Why was the index so much faster?	Without it, PostgreSQL reads all 600,000 rows. The B-tree seeks directly to job_id = 1234 and reads 3 blocks. Measured 62.6 ms vs 0.041 ms.
8	What is a partial index?	An index with a WHERE clause that covers only some rows (open applications, about 4%). It is smaller and cheaper to maintain.
9	How exactly does SQL injection work in your lab?	The unsafe code builds "... WHERE last_name = '" + input + "'". The input x' OR '1'='1 closes the quote and adds OR '1'='1', which is always true, so every row is returned.
10	If the unsafe endpoint is vulnerable, why is it safe to ship?	It runs as a role with SELECT on two tables only, in a READ ONLY transaction with a timeout, always rolled back, and the server binds to localhost. Every real query uses parameters.

6.5 Questions anyone may get

Question	Answer
Is the data real?	No. It is synthetic and seeded, with fictional names, so results are reproducible and nobody's data is exposed.
What would you do differently?	Row-level security per user, SERIALIZABLE isolation with retries, and materialized views for the dashboard.
What was the hardest part?	Getting the aggregation right so that interview and offer integrity could be enforced with keys, and the concurrency case.
Did you use any ORM?	No. Plain SQL with bound parameters, because the project is about SQL and the database does the work.
How did you test?	91 pytest tests against the real database, each in a rolled-back transaction, plus a two-connection concurrency test.

7. Personal preparation sheets

M1 · Ekta Sharma · Problem & EER

Item	Detail
Slides	1–5
Time window	0:00–3:45
Numbers to know	180 students, 16 companies, 15 business rules Constructs: overlapping (o), disjoint (d), weak entity, aggregation, union (U)
You must be able to explain	The three anomalies with the Northbridge example Why APPLIES_TO is aggregated Overlapping vs disjoint; total vs partial Category = Company ∪ Alumnus

Rehearse with the speaker notes in the slide deck (each slide's notes contain your script with timing).

M2 · [Member 2] · Schema & normalization

Item	Detail
Slides	6–8
Time window	3:45–7:30
Numbers to know	22 tables, 30 foreign keys FD1–FD9; Eligibility and Program come from 3NF
You must be able to explain	Mapping steps 1, 2, 5, 6, 8A, 9 Super / candidate / primary / alternate with placement_cycle Walk UNF → 1NF → 2NF → 3NF on the diagram Heath's theorem; the Panel 3NF-not-BCNF trap

Rehearse with the speaker notes in the slide deck (each slide's notes contain your script with timing).

M3 · [Member 3] · DDL, data & queries

Item	Detail
Slides	9–12
Time window	7:30–11:15
Numbers to know	45 CHECK, 17 UNIQUE, 1 EXCLUDE, 55 indexes 1,184 applications, 1,057 interviews, 96 offers Q2 → 8 students; Q5 → 3 students
You must be able to explain	Composite FK for disjointness (read the code) EXCLUDE with tstzrange && Data loaded through the triggers in one transaction Double NOT EXISTS; SUM() OVER running total

Rehearse with the speaker notes in the slide deck (each slide's notes contain your script with timing).

M4 · [Member 4] · Rules, transactions & performance

Item	Detail
Slides	13–18

Item	Detail
Time window	11:15–15:00
Numbers to know	1,527× faster; partial index 26× smaller 180 vs 0 rows in the injection lab 91/91 tests passed
You must be able to explain	The accept flow: lock → guard → update → cascade → commit Run the live demo (see the Demo Runbook) Two-session concurrency story and lock order Leading-column finding on PostgreSQL 18

Rehearse with the speaker notes in the slide deck (each slide's notes contain your script with timing).

8. Glossary

Term	Meaning
Aggregate (EER)	A relationship treated as an entity so it can join another relationship
BCNF	Every non-trivial FD has a super key on its left side
Bound parameter	A value sent separately from SQL text ($1), never parsed as SQL
Cascade (trigger)	The automatic follow-up changes made by trg_offer_accept_cascade
Composite key / FK	A key or foreign key made of more than one column
Deferred constraint	A check postponed until COMMIT, so related rows can be inserted in any order
Discriminator	A column (role_type, sponsor_type) telling which subtype a row belongs to
EXCLUDE constraint	Rejects rows that conflict under an operator, e.g. overlapping ranges
GiST	Index type supporting ranges and overlap searches
Heath's theorem	Splitting R on X → Y into (XY) and (R − Y) is lossless
Partial index	An index built only over rows matching a WHERE clause
Partial key	The attribute that distinguishes weak-entity instances of the same owner
Seed (setseed)	Fixes the random sequence so generated data is identical every run
SQLSTATE	Five-character standard error code, e.g. 23514 check violation
Surrogate key	An artificial identifier (job_id) used instead of a natural key
Window function	A function computed over a set of related rows without grouping them away

9. Numbers cheat sheet

Fact	Value
Tables / foreign keys / CHECK / UNIQUE	22 / 30 / 45 / 17
Triggers / views / routines	9 / 2 / 3
Students / companies / jobs	180 / 16 / 25
Applications / interviews / offers	1,184 / 1,057 / 96
Final cycle placed so far	30 of 177; median CTC ₹29 LPA; top ₹38 LPA
Q1 hardest recruiter	Meridian Insurance (8.6% offer rate)
Q2 talent match for job 25	8 students, led by DBA12-039 (Tanvi Jain)
Index speed-up (600k rows)	62.6 ms → 0.041 ms (1,527×)
Partial vs full index size	176 KB vs 4,488 KB
Injection lab	180 rows (unsafe) vs 0 rows (safe)
Tests	91 / 91 passed
Demo student / demo job	PGP62-084 (Dev Saxena, offer from DataSutra) / job 25 (Tidewater)